

Arquitectura de Historia de cuenta

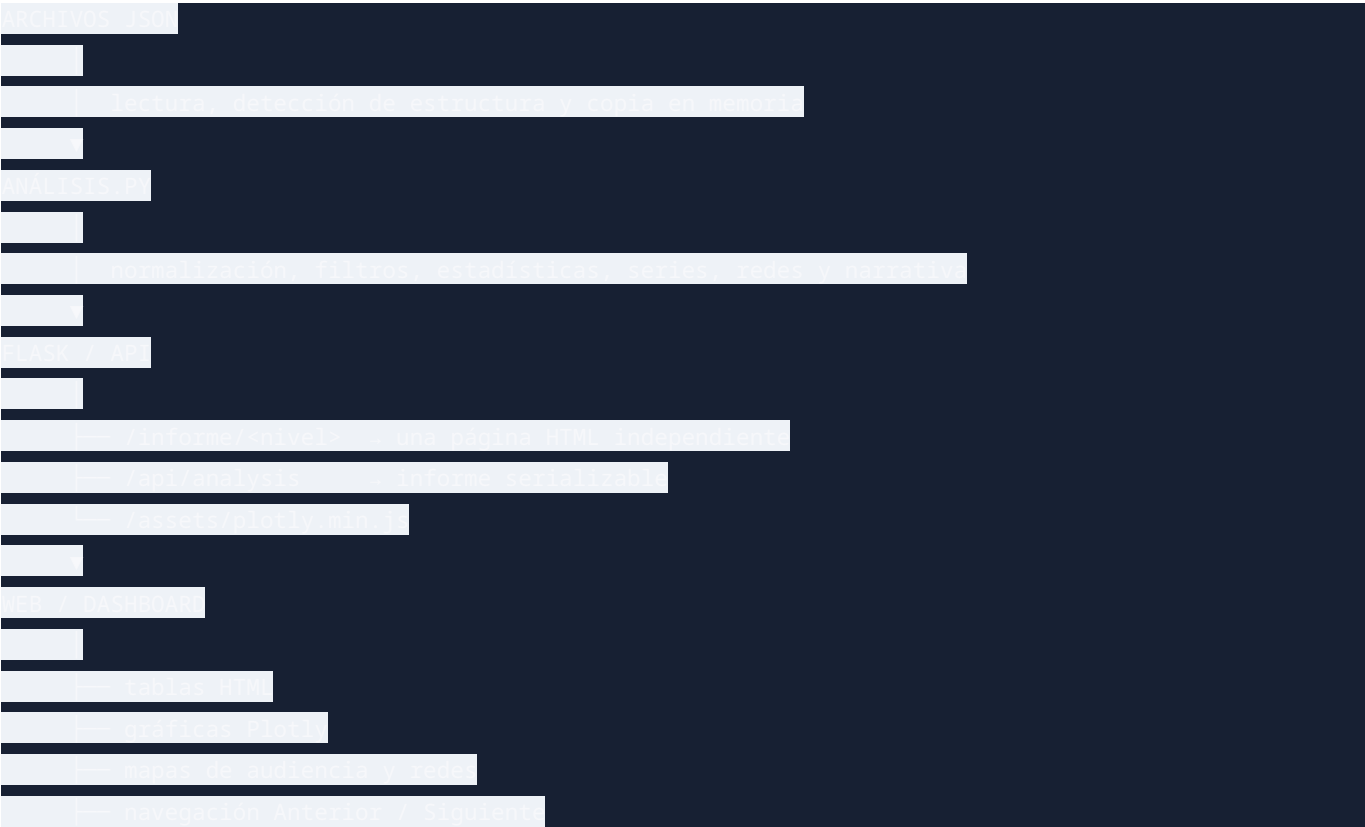
Documento técnico y funcional

Proyecto: Atlas / Historia de cuenta
Tipo de aplicación: informe web local para datos JSON de Instagram **Backend:** Python + Flask
Frontend: HTML, CSS y JavaScript con Plotly **Documento:** guía de arquitectura, funciones, modelos matemáticos y mantenimiento **Fecha de referencia:** 25 de septiembre de 2026

Resumen ejecutivo

La aplicación transforma uno o varios archivos JSON de Instagram en un informe visual. No intenta reemplazar a Instagram ni reconstruir información que el archivo no contiene. Su trabajo es organizar lo que sí está disponible, explicar cómo se calculó cada indicador y mostrar con claridad sus límites.

La arquitectura tiene cuatro capas principales:



La decisión central es separar **datos originales**, **datos derivados** y **presentación**. Los JSON no se modifican. El backend produce estructuras derivadas y el frontend las convierte en visuales sin tener que interpretar el JSON manualmente.

1. Objetivo y principios de diseño

1.1 Objetivo

La aplicación permite responder preguntas como:

- ¿Qué publicaciones hay en la fuente?
- ¿Qué publicaciones pertenecen a la cuenta principal?
- ¿Qué otros autores aparecen en los registros?
- ¿Qué likes y comentarios están disponibles?
- ¿Qué cuentas pueden atribuirse a interacciones?
- ¿Qué cuentas aparecen como colaboradoras?
- ¿Cómo se concentran las interacciones identificadas?
- ¿Qué cambios se observan por mes?
- ¿Qué redes y comunidades se observan?
- ¿Qué afirmaciones no se pueden hacer con estos datos?

1.2 Principios

1. **No modificar la fuente.** Los archivos se copian y se procesan en memoria.
2. **No inventar datos.** Si no hay identidades de likes, no se simulan personas.
3. **Separar hechos de interpretaciones.** Cada gráfica tiene una advertencia sobre lo que no permite concluir.
4. **Explicar los cálculos.** Los nombres técnicos se traducen a lenguaje claro.
5. **Mantener el estado simple.** No hay base de datos ni persistencia de informes.
6. **Preferir la lectura visual.** Cada nivel tiene una página HTML propia y las gráficas usan Plotly para ofrecer tooltips, zoom y responsividad.
7. **Mantener compatibilidad.** Se conservaron alias de respuestas y funciones antiguas para que herramientas existentes puedan seguir leyendo algunos campos.

1.3 Qué no hace

- No predice el alcance futuro.
- No identifica anonymously a personas que sólo aparecen en un número agregado.
- No demuestra que una colaboración cause likes.
- No demuestra que dos cuentas tengan una relación personal.
- No clasifica automáticamente una comunidad como amigos, familia, partido u organización.
- No corrige los sesgos de la descarga de Instagram.

2. Estructura del repositorio



2.1 `analysis.py`

Es el motor de datos. No depende de Flask y puede probarse por separado. Recibe una lista de posts, un objeto con posts o un sobre de varios archivos. Devuelve un diccionario serializable que contiene perfil, publicaciones, audiencia, redes, tablas, explicaciones y limitaciones.

2.2 `app.py`

Es la capa de servidor. Expone la API, carga los JSON iniciales, mantiene el informe actual en memoria, sirve las páginas HTML, entrega el JavaScript local de Plotly y permite activar autenticación básica mediante variables de entorno.

2.3 `web/index.html` y `web/landing.js`

Definen la pantalla de carga. `landing.js` valida los archivos en el navegador, envía un sobre a `/api/analysis` o usa `/api/load-server`, y abre `/informe/resumen` cuando el servidor ya tiene una fuente válida.

2.4 `web/pages/`

Contiene ocho páginas HTML independientes. Cada archivo tiene su propio título, navegación de nivel y contenido; el JavaScript común la completa con el informe y las gráficas. Las rutas son `/informe/resumen`, `/informe/datos`, `/informe/publicaciones`, `/informe/audiencia`, `/informe/colaboraciones`, `/informe/redes`, `/informe/evolucion` y `/informe/hallazgos`.

2.5 `web/dashboard.js`

Es el controlador del frontend actual. Controla la carga de la página, el selector de cuenta y alcance, la navegación, las tablas, el retrato narrativo y todas las figuras Plotly. No calcula métricas: consume el informe de `analysis.py`.

2.6 `web/dashboard.css`

Define el sistema visual responsive, la tipografía ampliada, tarjetas, tablas, leyendas, botones de navegación y los estados de carga y error.

2.7 `web/index.html` (compatibilidad anterior)

Define la estructura accesible del producto: pantalla de carga, navegación, secciones, tarjetas, tablas, gráficas, redes y cierre. Es la capa semántica; no calcula nada. Se conserva como compatibilidad y no es la ruta principal del dashboard actual.

2.8 `web/styles.css` (compatibilidad anterior)

Define el sistema visual: colores, tipografía, tarjetas, distribución responsive, tablas, gráficas, redes, estados vacíos y modalidades de lectura. No contiene lógica de análisis.

2.9 `web/app.js` (compatibilidad anterior)

Es el controlador del frontend heredado. Mantiene el estado visual, consume la API, dibuja gráficas SVG, actualiza tablas, controla eventos y genera las explicaciones visibles. Las páginas nuevas usan `dashboard.js` y Plotly.

2.10 Módulos JavaScript secundarios

`web/js/charts.js`, `web/js/formatters.js`, `web/js/network-layout.js` y `web/js/network-camera.js` son módulos de la versión anterior y se conservan como compatibilidad y para las pruebas. El dashboard actual no los importa: utiliza sus propias funciones de dibujo en `web/app.js`. Esto evita acoplar el nuevo diseño a una disposición antigua, pero los módulos siguen siendo parte del repositorio y deben documentarse si se mantienen.

2.11 Tests

- `tests/test_analysis.py` prueba normalización, alcances, comentarios, campos y compatibilidad con los JSON disponibles.
- `tests/test_app.py` prueba el estado, las rutas, la carga de archivos y la entrega de assets.

2.12 JSON de ejemplo

Los archivos `Cesar.json`, `Daniela.json`, `Hader.json` y `Maira.json` son fuentes de datos de trabajo. Están ignorados para nuevas adiciones por `.gitignore`, pero ya forman parte del historial del proyecto. La aplicación nunca los modifica.

3. Flujo completo de una petición

3.0 Navegación por páginas

1. La portada valida y carga la fuente en memoria.
2. Flask recibe el sobre y responde a `/api/analysis`.
3. El navegador abre `/informe/resumen`.
4. Cada página HTML declara su nivel mediante `data-page` y carga `dashboard.js`.
5. El controlador obtiene el informe con la cuenta y el alcance elegidos.
6. Los botones **Anterior** y **Siguiente** son enlaces reales a otra página HTML; no hay que hacer scroll automático entre secciones de una SPA.
7. Al cambiar de nivel se conserva el estado del servidor y se vuelve a dibujar sólo la página solicitada.

3.1 Carga desde el navegador (interfaz actual)

1. La persona selecciona o arrastra uno o varios archivos.
2. `uploadFiles()` lee cada archivo con `File.text()`.
3. Se parsea cada JSON en el navegador para comprobar sintaxis.
4. Se construye un sobre:



5. Se envía un `POST` a `/api/analysis`.
6. El servidor valida, analiza y devuelve el informe.
7. En la interfaz actual, la respuesta se conserva en el estado de Flask y la página HTML solicita el nivel que necesita. El flujo de `renderAll()` queda reservado para la interfaz heredada.

3.2 Carga desde el servidor

1. `GET /api/status` lista los JSON presentes en la carpeta.
2. La persona pulsa **Usar archivos del servidor**.

3. `POST /api/load-server` lee los archivos.
4. Se crea un `DatasetBundle` en memoria.
5. El estado del servidor se reemplaza sólo después de validar.

3.3 Recálculo de alcance

Cuando cambia la cuenta o el alcance:

1. `loadAnalysis()` obtiene los valores del formulario.
2. Ejecuta `GET /api/analysis?main=...&scope=...`.
3. Flask llama a `DashboardState.analysis()`.
4. El motor vuelve a normalizar y analizar el `DatasetBundle` almacenado.
5. El frontend recibe un nuevo objeto y vuelve a renderizar.

3.4 Diagrama de estados



4. Capa de datos: `analysis.py`

4.1 Constantes y diccionarios

`SUPPORTED_SCOPES`

Contiene los tres alcances válidos: `all`, `owned` e `involving`. La función `_normalize_scope()` rechaza valores desconocidos para evitar resultados ambiguos.

`POST_CONTAINER_KEYS`

Incluye nombres habituales de envoltorios: `posts`, `data`, `items`, `results`, `records`, `feed`, `content`, `media` y otros. Permite explorar una estructura sin exigir que el archivo sea exactamente una lista.

`LIKE_LIST_KEYS`

Lista aliases que pueden contener cuentas que dieron like: `likers`, `likedBy`, `liked_by`, `likeUsers`, `usersWhoLiked`, `likedAccounts` y otros.

`COMMENT_LIST_KEYS`

Agrupar `latestComments`, `comments`, `commentedPosts` y aliases con snake case.

`COLLABORATION_LIST_KEYS`

Agrupar `coauthorProducers`, `coauthors`, `collaborators`, `collaborations` y otras formas de coautoría.

`FOLLOWER_KEYS`

Identifica listas de seguidores y cuentas relacionadas: `followers`, `following`, `followerAccounts` y `followersData`.

`KNOWN_FIELD_DESCRIPTIONS`

Relaciona nombres técnicos como `likesCount`, `timestamp` o `coauthorProducers` con una

explicación en español para el inventario de datos.

4.2 DatasetError

Es una excepción específica para fuentes que no pueden producir un informe válido. Por ejemplo:

- el JSON no contiene posts;
- la lista está vacía;
- la cuenta solicitada no existe;
- el alcance no es válido.

Permite que Flask devuelva un mensaje 400 en lugar de una excepción interna.

4.3 DatasetBundle

Es el contenedor de trabajo en memoria. tiene cuatro responsabilidades principales:

1. recibir y copiar archivos;
2. encontrar publicaciones;
3. deduplicarlas;
4. indexar eventos de likes, comentarios, colaboraciones y seguidores.

DatasetBundle.from_payload(payload, source_name)

Entrada

- `payload`: lista, objeto o sobre de archivos.
- `source_name`: nombre mostrado para la fuente.

Proceso

1. Llama a `_coerce_files()`.
2. Extrae posts con `_extract_post_records()`.
3. Añade metadatos internos como `_source_name`, `_source_index`, `_record_index` y `_source_names`.
4. Elimina duplicados por `_post_identity()`.
5. Asigna un código estable a cada publicación.

6. Ejecuta `_index_events()`.
7. Devuelve un bundle nuevo.

Salida

Un `DatasetBundle` con `posts`, `inventory` y mapas de eventos. El objeto original recibido por la función no se modifica porque se trabaja con `copy.deepcopy()`.

`DatasetBundle._index_events()`

Construye índices de eventos:

- `like_events_by_post`;
- `comment_events_by_post`;
- `collaboration_events_by_post`;
- `followers`.

Recorre cada archivo y llama a los walkers correspondientes. Luego recorre los arrays que ya vienen dentro de cada post (`likers`, `latestComments`, `coauthorProducers`) para no perder datos que no están en un archivo separado.

Al final actualiza el inventario con el número de eventos encontrados por archivo.

4.4 Funciones de descubrimiento de archivos y posts

`_coerce_files(payload, source_name)`

Acepta tres formas:

- un sobre `{"files": [...]}`;
- un sobre `{"datasets": [...]}`;
- un único valor JSON.

Si los datos de un archivo vienen como string, intenta parsearlos. Devuelve una lista normalizada de archivos con `name` y `payload`.

`Path_name(value)`

Extrae sólo el nombre final de una ruta. Es un helper de compatibilidad y normalización; no interactúa con

el disco.

`_extract_post_records(value, path)`

Recorre listas y diccionarios buscando registros que parecen publicaciones. Si encuentra un post, no vuelve a tratar sus `childPosts` como publicaciones independientes.

Devuelve tuplas:

`(post, ruta dentro del JSON)`

La ruta se usa para depuración e inventario.

`_looks_like_post(value)`

Decide si un diccionario tiene forma de post. Exige señales de contenido, métricas, medio o una combinación explícita de autor e identificador. Excluye registros que parecen eventos de like.

`_post_identity(post)`

Produce una clave de deduplicación usando, en orden, `id`, `post_id`, `shortCode`, `code` o `url`. Si no hay ninguno, usa archivo e índice.

`_post_code(post, fallback_index)`

Obtiene el código público de la publicación. Es el valor que se utiliza para relacionar una cuenta con un post.

`_post_reference_values(post)`

Devuelve todos los identificadores que pueden relacionar un post con un evento externo: `id`, `shortCode`, código, URL y `postId`.

`_resolve_post_key(event, post_keys)`

Busca en un evento una referencia a una publicación y la convierte al código interno del post. Si no encuentra coincidencia, devuelve `None`; no asigna eventos al azar.

4.5 Walkers de eventos

`_walk_like_events(...)`

Recorre una estructura buscando registros de like identificados. Reconoce nombres como `likerUsername`, `likedBy`, `liked`, `likedAt` y el contexto de una lista `likes`.

Devuelve eventos con:



No transforma `likesCount` en identidades.

`_walk_comment_events(...)`

Busca registros con texto y autor. Se utiliza tanto para `latestComments` como para archivos separados de comentarios. Evita duplicar el post principal como si fuera un comentario.

`_walk_collaboration_events(...)`

Detecta coautores o colaboradores explícitos y los vincula al post cuando existe una referencia.

`_walk_follower_records(...)`

Detecta listas de seguidores. Conserva si el registro declara interacción, like, comentario y post asociado. Una lista de seguidores por sí sola no se convierte en audiencia.

4.6 Funciones de identidad y normalización

`_owner_value(post)`

Obtiene el autor desde aliases como `ownerUsername`, `accountUsername`, `username` o un objeto `owner/user` anidado.

`_account_from_value(value)`

Obtiene un username desde una cadena o un objeto con campos como `username`, `ownerUsername`, `user`, `owner`, `liker`, `collaborator` o `coauthor`.

`_account_values(value)`

Descompone una lista, mapa o estructura de cuentas en valores individuales. Admite listas de strings y objetos.

`normalize_username(value)`

Normaliza un usuario para comparar:

- quita espacios;
- quita @ inicial;
- convierte a minúsculas;
- acepta URLs de perfil y extrae el final del path.

No cambia el valor original del JSON.

`_normalize_account_value(value)`

Normaliza tanto strings como diccionarios que contienen una cuenta.

4.7 Funciones de tipos, fechas y números

`_safe_int(value, allow_missing=True)`

Convierte un valor a entero no negativo. `-1`, `null`, valores no numéricos y vacíos se consideran faltantes cuando corresponde.

`_count(value)`

Convierte un valor a un conteo no negativo y devuelve cero si no es válido.

`_first_value(source, *keys)`

Devuelve el primer valor no vacío entre varios alias. Se utiliza para no repetir estructuras `a` `or b` `or c`.

`_first_string(source, *keys)`

Igual que `_first_value`, pero devuelve string o `None`.

`_safe_text(value, limit)`

Limpia espacios, reduce saltos de línea y limita la longitud de textos mostrados.

`_truthy(value)`

Interpreta booleanos, 1 y strings como `true`, `yes` o `1`.

`_parse_datetime(value)`

Acepta:

- ISO 8601;
- fechas simples;
- timestamps Unix en segundos o milisegundos;
- objetos `datetime` y `date`.

Devuelve un `datetime` con zona horaria cuando es posible.

`_month_key(value)`

Devuelve `YYYY-MM` para agrupar por mes.

`_week_key(value)`

Devuelve `YYYY-Www` según la semana ISO.

4.8 Estadística descriptiva

`_percentile(values, percentile)`

Calcula un percentil con interpolación lineal entre posiciones ordenadas.

`_round(value, digits)`

Redondea valores y transforma valores no finitos en `None` para que el JSON sea válido.

`_descriptive_stats(values)`

Devuelve:

```
count, sum, mean, median, stddev, min, max,  
p25, p75, idk, ps
```

Los valores `None` se excluyen.

`_histogram(values, bins)`

Divide el rango de likes en ocho grupos, calcula conteos y devuelve inicio, final, etiqueta y cantidad.

`_boxplot(values)`

Calcula:

- mínimo;
- P25;
- mediana;
- P75;
- máximo;
- bigotes;
- valores atípicos usando rango intercuartílico.

4.9 Concentración y correlación

`_gini(values)`

Calcula el índice de Gini para valores positivos ordenados.

```
i = 2 * (G * n) / (n + 1)
```

Si sólo hay una cuenta, devuelve 1.0. Si no hay total, devuelve `None`.

`_lorenz(values)`

Construye los puntos acumulados de la curva de Lorenz. El primer punto es `(0, 0)`.

`_rank(values)`

Calcula rangos promedio para empates. Es la base de Spearman.

`_correlation(xs, ys, method)`

Calcula Pearson o Spearman. Necesita al menos tres observaciones y que ninguna variable sea constante.

`_correlations(posts, audience)`

Prepara comparaciones como:

- likes y comentarios;
- likes y colaboradores;
- likes y visualizaciones;
- frecuencia mensual y likes;
- interacciones y colaboraciones por cuenta.

Devuelve coeficientes, tamaño de muestra, fuerza e interpretación prudente.

`_correlation_strength(value)`

Clasifica el valor absoluto:

- menor que 0,2: muy débil;

- 0,2–0,4: débil;
- 0,4–0,6: moderada;
- 0,6–0,8: fuerte;
- 0,8 o más: muy fuerte.

4.10 Cuentas y recurrencia

`_new_account(username)`

Crea el estado interno de una cuenta:

- likes;
- comentarios;
- interacciones;
- menciones;
- etiquetas;
- colaboraciones;
- conjuntos de posts;
- primera y última aparición;
- meses activos;
- extractos de comentarios.

Los `set` se usan internamente y se convierten a listas al serializar.

`_merge_account_metadata(account, source)`

Completa nombre, ID, foto y verificación sin sobrescribir información con valores vacíos.

`_update_appearance(account, timestamp, post_code, post_date)`

Actualiza posts relacionados, meses activos, primera aparición y última aparición.

`_serialize_account(account)`

Convierte el estado interno en un diccionario compatible con JSON. Es donde se calculan los alias `publications_with_interaction`, `combined_score` y `recurrence`.

`_recurrence_label(interactions, months, span_days)`

Aplica las reglas:

- `Ocasional`: una interacción o un mes;
- `Persistente`: tres o más interacciones y al menos tres meses o 60 días;
- `Recurrente`: el resto.

`_iter_dates(posts)`

Extrae fechas válidas. Es una utilidad de apoyo para posibles cálculos temporales.

`_period_series(posts, period)`

Agrupar por mes o semana y calcula:

- publicaciones;
- likes conocidos;
- publicaciones con likes faltantes;
- comentarios reportados;
- comentarios capturados;
- cuentas con interacción;
- colaboraciones;
- publicaciones colaborativas.

`_new_account_for_relation(username, accounts)`

Obtiene o crea una cuenta en el diccionario de relaciones.

`_metadata_for_username(posts, username)`

Busca metadatos de una cuenta en los posts. Es un helper de compatibilidad para nombres de cuenta.

4.11 Normalización de publicaciones

`_post_type(post)`

Obtiene el tipo técnico y aplica alias como `mediaType` y `productType`.

`_content_category(post)`

Mapea el tipo técnico a categorías de lectura: Imagen, Vídeo, Reel, Carrusel u Otro. No interpreta el tema.

`_post_metric(post, *keys)`

Obtiene el primer campo numérico válido entre alias como `likesCount`, `likeCount` y `like_count`.

`_build_normalized_posts(bundle, selected_posts, main_account)`

Es una de las funciones centrales. Para cada post seleccionado:

1. obtiene likes, comentarios y fecha;
2. deduplica eventos;
3. agrega likes y comentarios por cuenta;
4. calcula `interaction_counts`;
5. extrae menciones, etiquetas y colaboradores;
6. construye el post normalizado;
7. conserva el nombre del archivo fuente.

El post normalizado contiene datos para la tabla, las gráficas, las redes, la matriz y las explicaciones.

`_dedupe_events(events, identity_key)`

Elimina duplicados de eventos por cuenta, identificador y timestamp. Para likes, además se evita atribuir dos veces el mismo like de la misma cuenta al mismo post.

4.12 Filas de audiencia y matriz

`_build_account_rows(accounts)`

Serializa y ordena cuentas por interacciones, posts y username.

`_build_audience(account_rows)`

Conserva únicamente cuentas con interacciones identificadas y agrega alias como `number_of_interactions`.

`_audience_matrix(posts, audience)`

Construye la matriz:

fila = cuenta	
columna = número de interacciones identificadas	

Limita la matriz a 40 cuentas y 60 posts para mantenerla utilizable en el navegador. `truncated` indica si se recortó.

4.13 Redes

`_build_star_network(rows, main_account, kind, weight_key)`

Construye una red central con un nodo para la cuenta principal y nodos periféricos para cuentas cuyo `weight_key` es positivo. El peso de la arista es el número de interacciones o colaboraciones.

`_build_cooccurrence_network(posts, main_account)`

Genera pares de cuentas que aparecen en la misma publicación. El peso es el número de publicaciones compartidas.

`_graph_statistics(nodes, edges)`

Calcula:

- número de nodos y aristas;

- componentes conectados;
- componente mayor;
- densidad;
- peso máximo;
- modularidad.

`_connected_components(nodes, edges)`

Implementa recorrido BFS/DFS para encontrar componentes de una gráfica no dirigida.

`_centralities(nodes, edges)`

Calcula:

- grado;
- grado ponderado;
- betweenness.

`_betweenness(nodes, adjacency)`

Implementa el algoritmo de Brandes simplificado para la intermediación de caminos más cortos. El valor se normaliza dividiendo entre $n - 1$.

`_k_core(nodes, edges)`

Elimina nodos con pocos vínculos y conserva el núcleo más densamente conectado.

`_bridge_candidates(nodes, edges)`

Ordena cuentas no centrales por betweenness y grado. Una cuenta puente es una posición estructural, no una persona “importante” por definición.

`_context_network(posts)`

Mantiene la red de contexto heredada para lugares y música. Aunque el dashboard actual se centra en cuentas, el payload conserva este bloque por compatibilidad.

4.14 Comparaciones, tipos y hallazgos

`_build_comparison(posts)`

Separa publicaciones con y sin colaboración y calcula total, promedio, mediana, desviación, máximo, mínimo y likes por post.

`_content_summary(posts)`

Agrupar publicaciones por categoría técnica y calcula frecuencia, likes, colaboración y cuentas de interacción.

`_build_exceptions(posts, stats)`

Detecta publicaciones con likes por encima de $P75 + 1,5 \cdot IQR$ o con z-score de dos o más. Devuelve también posición relativa y cuentas que interactuaron.

`_build_findings(...)`

Construye entre 5 y 10 hallazgos. Cada hallazgo tiene:

id	
likes	
comments	
followers	
collaborations	

Las afirmaciones se generan a partir de indicadores disponibles; no se inventan patrones.

`_data_coverage(...)`

Indica si hay identidades de likes, comentarios, seguidores y colaboraciones. Distingue eventos de la fuente y eventos del alcance seleccionado.

`_cross_file_links(bundle)`

Compara identificadores de post, usuarios y fechas entre archivos para sugerir cruces.

`_describe_file(...)`

Construye el inventario de un archivo:

- rol inferido;
- cantidad de posts;
- campos;
- identificadores;
- notas.

`_collect_fields(...)`

Recorre el JSON hasta una profundidad limitada y registra rutas de campos. Evita recorrer indefinidamente estructuras muy grandes.

`_inventory(bundle, posts)`

Construye el bloque `data_inventory`, que la interfaz muestra como “Qué datos tenemos”.

`_limitations(...)`

Genera advertencias explícitas sobre cobertura, likes agregados, relaciones y archivos sin posts.

4.15 Alcance y funciones públicas

`_normalize_scope(scope)`

Valida `owned`, `involving` o `all`.

`_post_involves_main(post, main_account)`

Devuelve verdadero si la cuenta principal es autora o aparece en menciones, etiquetas o colaboración.

`select_posts(posts, main_account, scope)`

Aplica el alcance. Las funciones de análisis utilizan esta función para no duplicar la lógica.

`determine_main_account(posts, requested)`

Cuenta autores y elige el más frecuente, salvo que se solicite una cuenta concreta.

_full_name_for_main(posts, username)

Busca el nombre público de la cuenta principal.

analyze_payloads(payloads, main_account, scope, source_name)

Es la API principal del motor. Coordina todo el proceso y devuelve el informe completo.

_post_reference(post)

Crea el resumen de la publicación con más likes y con menos likes.

_post_table_row(post, stats)

Prepara una fila de tabla con fecha, publicación, likes, colaboración, colaborador, tipo y posición relativa.

_follower_rows(followers)

Resume seguidores únicos y seguidores que interactúan.

_recurrence_summary(audience)

Construye las tres categorías de recurrencia.

analyze_posts(posts, ...)

Mantiene la interfaz antigua de una lista de posts. Envuelve la lista en un archivo y llama a `analyze_payloads()`.

5. Capa Flask: app.py

5.1 DashboardState

Es el estado temporal del servidor. Se instancia una vez como `state`.

`__init__(posts, source_name)`

Inicializa posts, nombre de fuente, lock y cuenta principal.

`replace_dataset(posts, source_name)`

Alias compatible para reemplazar una lista de posts.

`replace_payload(payload, source_name)`

Construye y valida un `DatasetBundle` antes de asignarlo. Si la fuente nueva es inválida, el informe anterior permanece intacto.

`replace_files(files)`

Carga un conjunto de archivos como un sobre.

`clear()`

Borra el estado en memoria.

`current_bundle()`

Devuelve el bundle actual. Si una prueba o una herramienta modificaron directamente `state.posts`, reconstruye el bundle.

`analysis(main_account, scope)`

Pasa el bundle a `analyze_payloads()`.

`status()`

Devuelve si hay datos, fuente, cuenta principal y cantidad de posts.

5.2 Autenticación

`check_auth(username, password)`

Compara las credenciales sólo si `AUTH_ENABLED` es verdadero.

`authenticate()`

Construye la respuesta 401 con `WWW-Authenticate: Basic`.

`require_login()`

Middleware `before_request`. Permite `/api/health` sin autenticación y protege el resto cuando se configuraron `ADMIN_USER` y `ADMIN_PASSWORD`.

La autenticación no se muestra como un campo de contraseña dentro del dashboard. Se configura en el servidor para no exponerla en la interfaz.

5.3 Helpers del servidor

`_query_value(name, default)`

Lee un parámetro de query string.

`_decode_payload(raw)`

Decodifica UTF-8 con BOM opcional y parsea JSON. Convierte errores en `DatasetError`.

`_load_server_files()`

Lee todos los `.j` son de la carpeta base sin escribir en ellos. Omite archivos inválidos.

5.4 Rutas Flask

GET /api/health

Comprueba que el servidor está vivo.

GET /api/status

Devuelve estado actual, archivos disponibles, si la autenticación está activa y un mensaje de uso.

GET|POST /api/analysis

- En POST, valida un archivo o sobre, calcula el informe y reemplaza el estado sólo después del análisis correcto.
- En GET, recalcula la fuente actual con `main` y `scope`.

POST /api/load-server

Carga todos los JSON del servidor en memoria y devuelve el informe.

GET /api/sources

Lista archivos y tamaños.

GET /assets/plotly.min.js

Entrega el JavaScript de Plotly desde el paquete local. La respuesta se memoriza para no regenerar el bundle en cada petición.

GET /informe/<page>

Entrega una de las ocho páginas HTML independientes. `PAGE_FILES` limita los nombres válidos y evita que la ruta se convierta en una lectura arbitraria del sistema.

GET /GUIA_DE_ANALISIS.md

Entrega la guía de datos y cálculos como Markdown.

`serve_static(path)`

Sirve `index.html`, CSS, JavaScript, Plotly local y las páginas HTML. Las rutas `/informe/<page>` se resuelven antes para entregar el HTML del nivel solicitado.

5.5 Compatibilidad y ejecución

`find_default_dataset()`

Busca el primer `.json` del proyecto. Se conserva para las pruebas antiguas.

`make_handler(dataset_state)`

Crea un `BaseHTTPRequestHandler` mínimo para probar el estado sin WSGI. No es el servidor principal de producción.

`_parse_args()`

Define `--dataset`, `--main`, `--scope` y `--port`.

El bloque `if __name__ == "__main__"` carga opcionalmente un dataset inicial y arranca Flask.

6. Capa de presentación

La interfaz vigente se divide en una portada y ocho páginas HTML. La sección 6.1 describe la portada; las páginas de nivel y el controlador Plotly se documentan a continuación. Las subsecciones del panel antiguo se conservan como compatibilidad.

6.1 `web/index.html` y `web/landing.js`

La portada sólo carga la fuente. `landing.js` lee uno o varios archivos, comprueba el límite de 100 MB, construye un sobre `files` y hace POST `/api/analysis`. Si ya existe estado, muestra un enlace para continuar. No calcula ninguna métrica.

6.2 Páginas HTML de nivel

Cada archivo bajo `web/pages/` tiene un `data-page`, una cabecera propia, selectores de cuenta/alcance, un contenedor `#pageContent` y enlaces de navegación. El servidor los entrega en `/informe/<page>`; no se renderizan todas las secciones en un documento largo.

La secuencia es:



6.3 `web/dashboard.js`

El controlador común obtiene el informe y selecciona una función de render según `document.body.dataset.page`. `renderResumen`, `renderDatos`, `renderPublicaciones`, `renderAudience`, `renderColaboraciones`, `renderNetworks`, `renderEvolution` y `renderFindings` construyen el HTML de esa página.

Las funciones `barPlot`, `linePlot`, `boxPlot`, `boxGroupsPlot`, `heatmapPlot` y `networkPlot` entregan trazas a `window.Plotly.newPlot`. El ranking de publicaciones usa etiquetas `Publicación 1`, `Publicación 2`, etc., y mantiene una lista de enlaces reales junto al histograma. La línea temporal separa likes de la actividad editorial. No se usa Matplotlib. Plotly se carga desde `/assets/plotly.min.js`, por lo que la aplicación no necesita un CDN.

6.4 `web/dashboard.css`

El sistema visual usa una base tipográfica de 16 px, títulos grandes, tarjetas con contraste, botones de 44 px, tablas responsive y una leyenda de cuatro colores en Evolución. Las páginas se adaptan a una columna en móvil.

6.5 `profile_narrative`

`analysis.py` añade un bloque narrativo al informe. No es un diagnóstico: seis dimensiones (ritmo, contenido, respuesta, conversación, colaboración y red) convierten los datos en una lectura de la cuenta, pero cada una muestra su evidencia y su límite. La página de Hallazgos lo presenta antes de los hallazgos estadísticos.

6.6 Panel heredado

`web/index.html` (compatibilidad anterior)

El documento se divide en:

1. pantalla de carga;
2. barra superior;
3. selector de cuenta y alcance;
4. resumen;
5. datos disponibles;
6. publicaciones;
7. audiencia;
8. colaboraciones;
9. redes;
10. evolución;
11. hallazgos y límites.

Cada gráfica tiene un contenedor con `id`. JavaScript lo busca con `$( )` y reemplaza su contenido. Esta separación permite que HTML sea estable y que el código visual sea dinámico.

6.7 Estado de `web/app.js`

El objeto `state` contiene:

<code>data</code>	informe actual
<code>reportStatus</code>	estado del servidor
<code>network</code>	red seleccionada
<code>selectedAccount</code>	cuenta seleccionada
<code>audienceSearch</code>	filtro de cuenta
<code>loading</code>	estado de carga
<code>searchInput</code>	campo de búsqueda de la red

6.8 Funciones de formato y seguridad

`formatNumber(value)`

Formatea números con separador de miles español.

formatCompact(value)

Usa notación compacta para ejes, por ejemplo 1,2 mil.

formatPercent(value)

Formatea proporciones como 35,0 %.

escapeHtml(value)

Escapa &, <, >, ' y " antes de insertar texto generado.

safeUrl(value)

Sólo permite URLs http y https. Evita convertir un campo de JSON en un enlace javascript:.

normalizeText(value)

Quita tildes y pasa a minúsculas para búsquedas.

formatDate(value, withTime)

Formatea fecha y, opcionalmente, hora.

formatMonth(value)

Convierte YYYY-MM en una etiqueta corta.

6.9 Feedback y API

showToast(message, error)

Muestra un mensaje temporal.

setLoading(value, title, text)

Muestra u oculta la pantalla de carga.

parseResponse(response)

Convierte la respuesta en JSON y convierte errores HTTP en mensajes comprensibles.

setUpUploadStatus(message, type)

Actualiza el mensaje de la pantalla inicial.

showUpload() y **showReport()**

Cambian entre portada y dashboard.

loadAnalysis()

Solicita un nuevo informe para la cuenta y alcance actuales.

uploadFiles(fileList)

Valida extensión, lee archivos, crea el sobre y hace POST.

loadServerFiles()

Llama a `/api/load-server`.

6.5 Orquestación del render

renderAll()

Ejecuta, en orden:



renderHeader()

Muestra cuenta, periodo, alcance, total de likes, cobertura de datos y advertencia sobre likers.

renderKpis()

Construye las tarjetas: publicaciones, likes, cuentas, colaboraciones, seguidores y periodo.

renderDataInventory()

Dibuja tarjetas por archivo, roles, campos y un diccionario de variables.

6.6 Gráficas SVG

chartSvg(options)

Envuelve el contenido de una gráfica en un `<svg>` con `viewBox` y `aria-label`.

gridLines(...)

Genera líneas horizontales y etiquetas del eje vertical.

verticalBars(...)

Dibuja barras verticales, Tooltip y etiquetas de eje.

lineChart(items, series, options)

Dibuja varias líneas sobre los mismos periodos. Cada serie tiene `key` y `color`.

scatterPerformance(posts)

Dibuja la relación temporal entre publicaciones y likes. El color de colaboración se muestra como punto colaborativo.

renderLikesByPost()

Ordena publicaciones por fecha y dibuja likes por post.

renderLikesTimeline()

Dibuja la serie mensual de likes conocidos.

renderPostFrequency()

Dibuja publicaciones por mes.

renderLikesHistogram()

Dibuja el histograma.

renderBoxplot()

Dibuja P25, mediana, P75, bigotes y valores atípicos.

renderPerformance()

Conecta la dispersión con el panel de rendimiento.

renderExceptions()

Muestra tarjetas de publicaciones excepcionales.

renderContentTypes()

Dibuja publicaciones por categoría técnica.

6.7 Audiencia

renderAudience()

Actualiza resumen, tabla, mapa, concentración, Lorenz, recurrence y heatmap.

renderConcentration()

Dibuja barras del top 5, top 10 y top 20.

renderLorenz()

Dibuja diagonal y curva acumulada.

renderRecurrence()

Dibuja las tres categorías.

renderAudienceHeatmap()

Construye una cuadrícula HTML de celdas con intensidad según número de interacciones.

6.8 Redes

networkColor(index)

Genera colores cíclicos para distinguir nodos.

renderNetworkSvg(...)

Dibuja nodos, aristas, títulos y etiquetas. La cuenta principal ocupa el centro en las redes centrality.

renderAudienceNetwork()

Dibuja el mapa de audiencia.

renderCollaborations()

Actualiza ranking de colaboradores, serie mensual, comparación y matriz de presencia.

renderNetworks()

Cambia entre red de interacción, colaboración y coincidencia; actualiza leyenda, comunidades y puentes.

6.9 Evolución y hallazgos

`renderEvolution()`

Dibuja las líneas de la cuenta y de la audiencia.

`renderFindings()`

Genera tarjetas de evidencia y cautions. Añade correlaciones si existen.

`setExplanation(key, shows, observes, caution)`

Escribe las tres preguntas de cada gráfica:

¿Qué pasó?
¿Qué podemos observar?
¿Qué no podemos concluir?

`renderExplanations()`

Personaliza el texto de cada gráfica con los valores del informe actual.

`downloadCsv()`

Descarga la tabla de publicaciones.

`downloadNetworkCsv()`

Descarga nodos y aristas de la red actual como CSV, compatible con una importación sencilla en Gephi.

`bindEvents()`

Conecta upload, drag-and-drop, filtros, submit, pestañas, scrolls y botones.

`initialize()`

Registra eventos, consulta `/api/status`, muestra los archivos del servidor y deja visible la portada.

7. Módulos JavaScript secundarios

Aunque el dashboard actual no los importa, se mantienen como módulos de compatibilidad.

web/js/formatters.js

- `formatNumber`: separador de miles.
- `formatPercent`: porcentaje.
- `formatCompact`: compacto.
- `formatDate`: fecha local.
- `formatMonthRange`: intervalo de meses.
- `escapeHtml`: escape de HTML.
- `normalizeSearch`: búsqueda sin tildes.
- `profileUrl`: URL de perfil.
- `postUrl`: URL de publicación.
- `initials`: iniciales para avatar.

web/js/charts.js

- `monthLabel`: etiqueta de mes.
- `barChart`: barras SVG.
- `updateSummary`: resumen de una serie.
- `renderMonthlyCharts`: render de posts, likes y colaboraciones.

web/js/network-layout.js

- `clamp`: limita un valor.
- `connectedComponents`: componentes de una red.
- `componentTargets`: distribuye componentes en el espacio.
- `recenterComponent`: centra un componente.
- `layoutSecondaryNetwork`: layout de red secundaria.
- `layoutContextNetwork`: layout de contexto.

- `layoutEgoNetwork`: layout radial de la cuenta central.

`web/js/network-camera.js`

- `clamp`: limita el zoom.
- `createNetworkCamera`: gestiona zoom, rueda, arrastre y viewBox.

8. Modelos matemáticos y estadísticos

8.1 Modelo de publicación

La unidad de análisis es una publicación deduplicada. Su vector de observación puede verse así:



8.2 Modelo de cuenta

Cada cuenta se representa con agregados y conjuntos:



8.3 Modelo de red

Una red es una gráfica no dirigida:



- V: nodos, normalmente cuentas;
- E: vínculos observados.

La red de interacción y colaboración son redes estrella. La red de coincidencia es una red de co-ocurrencia entre cuentas periféricas.

8.4 Modelo de concentración

Se ordenan las interacciones por cuenta y se calculan los grupos más recurrentes, Gini y Lorenz.

8.5 Modelo de correlación

Pearson y Spearman son modelos bivariados de asociación. No son modelos causales. No se utilizan para predecir likes ni para recomendar contenido.

8.6 Por qué no hay un modelo predictivo

El sistema no incorpora machine learning, regresión forecasting ni clasificación de comunidades con redes neuronales. La fuente puede ser pequeña, sesgada y parcial. Un modelo predictivo daría una precisión aparentemente avanzada, pero podría inducir a interpretar como certeza algo que sólo es una muestra.

La aplicación prefiere mostrar:

- qué se observó;
- cuánto;
- dónde;
- con qué cobertura;
- qué falta.

9. Seguridad, privacidad y datos originales

9.1 Datos originales

Los JSON se copian en memoria. No se crea una carpeta de resultados junto a ellos. El nombre del archivo se conserva como metadato de fuente.

9.2 Datos personales

Los usernames y nombres públicos se muestran porque aparecen en los registros. El informe debe tratarse como información de una descarga concreta, no como un censo completo de Instagram.

9.3 Autenticación

Para activar la protección básica:

```
ADMIN_USERNAME=admin ADMIN_PASSWORD=contaseña python app.py
```

No se recomienda reutilizar una contraseña personal. En producción se debe utilizar HTTPS, un servidor WSGI como Gunicorn y un gestor de secretos.

9.4 Límite de subida

Flask limita el cuerpo a 100 MB. El usuario puede cargar varios archivos, pero el conjunto debe caber en ese límite.

9.5 Escape de contenido

El frontend escapa texto y valida URLs antes de insertarlas en HTML. Esto evita que un caption o URL almacenado en JSON se convierta en una instrucción HTML.

10. Pruebas y mantenimiento

Ejecutar pruebas

```
python -m unittest discover -s tests
```

Comprobación de JavaScript

```
node --check web/dashboard.js
```

```
node --check web/landing
```

Comprobación de Python

```
python -m py_compile analysis.py app.py
```

Qué se prueba

- normalización de nombres de usuario;
- selección de cuenta;
- alcances;
- comentarios y cuentas;
- contexto;
- JSON disponibles;
- carga de servidor;
- carga transaccional;
- estado inicial;
- rutas de assets;
- páginas `/informe/<page>` y navegación;
- disponibilidad de Plotly local;
- presencia de `profile_narrative` y sus límites.

Qué falta como prueba futura

- fixtures con likes separados;
- fixtures con seguidores;
- duplicados entre archivos;
- publicaciones sin fecha;
- campos `-1`;
- gráficas de red con componentes;
- pruebas end-to-end del navegador.

11. Decisiones de arquitectura y alternativas

Opción elegida: backend Python sin ORM

Se eligió el procesamiento estándar porque los JSON son pequeños y se procesan en memoria. Un ORM no es necesario y complicaría el estado temporal.

Opción elegida: frontend sin framework

Se eligió JavaScript nativo para que el producto sea fácil de desplegar y no dependa de Node en ejecución. El estado es pequeño y se gestiona con un objeto `state`.

Opción elegida: Plotly

Plotly se distribuye como JavaScript local desde la dependencia Python `plotly`. Ofrece barras, líneas, dispersión, cajas, heatmaps y redes interactivas sin usar Matplotlib ni depender de un CDN. El bundle se sirve desde `/assets/plotly.min.js` y se memoriza en el servidor.

Alternativa: React/Vue

Sería conveniente si el dashboard creciera con muchas vistas, filtros persistentes y estado complejo. Para el diseño actual, habría más configuración y build de la que se necesita.

Alternativa: NetworkX + Gephi

NetworkX podría simplificar la generación de redes, pero la implementación actual usa grafos estándar para reducir dependencias. El CSV permite abrir la red en Gephi si se necesita análisis externo.

Alternativa: base de datos

Una base de datos sería útil para historial, usuarios y dashboards comparables. No es necesaria para el modo local actual.

12. Índice de funciones

Esta lista funciona como mapa de lectura del código.

Backend analysis.py

Función o clase	Responsabilidad
<code>DatasetError</code>	Error de fuente inválida.
<code>DatasetBundle</code>	Copia, deduplicación e indexación de fuentes.
<code>DatasetBundle.from_payload</code>	Acepta diferentes estructuras de archivo.
<code>DatasetBundle._index_events</code>	Indexa likes, comentarios, colaboraciones y seguidores.
<code>_coerce_files</code>	Normaliza sobres y archivos.
<code>Path_name</code>	Extrae nombre de ruta.
<code>_extract_post_records</code>	Encuentra posts.
<code>_looks_like_post</code>	Reconoce un post.
<code>_post_identity</code>	Crea clave de deduplicación.
<code>_post_code</code>	Obtiene código de post.
<code>_post_reference_values</code>	Obtiene referencias de post.
<code>_resolve_post_key</code>	Relaciona evento con post.
<code>_walk_like_events</code>	Busca likes identificados.
<code>_walk_comment_events</code>	Busca comentarios.
<code>_walk_collaboration_events</code>	Busca colaboradores.
<code>_walk_follower_records</code>	Busca seguidores.
<code>_owner_value</code>	Obtiene autor.
<code>_account_from_value</code>	Obtiene cuenta.
<code>_account_values</code>	Descompone cuentas.

Función o clase	Responsabilidad
<code>normalize_username</code>	Normaliza usuario.
<code>_normalize_account_value</code>	Normaliza cuenta u objeto.
<code>_safe_int</code>	Entero seguro.
<code>_count</code>	Conteo seguro.
<code>_first_value</code>	Primer alias.
<code>_first_string</code>	Primer string.
<code>_safe_text</code>	Texto limpio.
<code>_truthy</code>	Booleano tolerante.
<code>_parse_datetime</code>	Parsea fecha.
<code>_month_key</code>	Agrupar por mes.
<code>_week_key</code>	Agrupar por semana.
<code>_percentile</code>	Percentil.
<code>_round</code>	Redondeo seguro.
<code>_descriptive_stats</code>	Estadísticas.
<code>_histogram</code>	Histograma.
<code>_boxplot</code>	Caja y bigotes.
<code>_gini</code>	Índice Gini.
<code>_lorenz</code>	Curva Lorenz.
<code>_rank</code>	Rangos para Spearman.
<code>_correlation</code>	Pearson o Spearman.
<code>_correlations</code>	Comparaciones estadísticas.

Función o clase	Responsabilidad
<code>_correlation_strength</code>	Etiqueta de fuerza.
<code>_new_account</code>	Estado de cuenta.
<code>_merge_account_metadata</code>	Metadata de cuenta.
<code>_update_appearance</code>	Fechas y posts de cuenta.
<code>_serialize_account</code>	Serialización de cuenta.
<code>_recurrence_label</code>	Categoría de recurrencia.
<code>_iter_dates</code>	Fechas extraídas.
<code>_period_series</code>	Series mensual/semanal.
<code>_new_account_for_relation</code>	Obtiene/crea cuenta.
<code>_metadata_for_username</code>	Busca metadata.
<code>_post_type</code>	Tipo técnico.
<code>_content_category</code>	Categoría técnica.
<code>_post_metric</code>	Métrica numérica.
<code>_build_normalized_posts</code>	Normaliza publicaciones y eventos.
<code>_dedupe_events</code>	Elimina duplicados.
<code>_build_account_rows</code>	Filas de cuentas.
<code>_build_audience</code>	Filas de audiencia.
<code>_audience_matrix</code>	Matriz cuenta/post.
<code>_build_star_network</code>	Red central.
<code>_build_cooccurrence_network</code>	Red de coincidencia.

Función o clase	Responsabilidad
<code>_graph_statistics</code>	Estadísticas de grafo.
<code>_connected_components</code>	Componentes.
<code>_centralities</code>	Centralidad.
<code>_betweenness</code>	Intermediación.
<code>_k_core</code>	Núcleo.
<code>_bridge_candidates</code>	Puentes.
<code>_context_network</code>	Lugares y música.
<code>_build_comparison</code>	Colaborativas vs. no colaborativas.
<code>_content_summary</code>	Resumen por tipo.
<code>_build_exceptions</code>	Valores atípicos.
<code>_build_findings</code>	Hallazgos con evidencia y advertencia.
<code>_build_profile_narrative</code>	Retrato descriptivo de la cuenta.
<code>_data_coverage</code>	Cobertura.
<code>_cross_file_links</code>	Cruces entre archivos.
<code>_describe_file</code>	Inventario de archivo.
<code>_collect_fields</code>	Rutas de campos.
<code>_inventory</code>	Inventario completo.
<code>_limitations</code>	Límites.
<code>_normalize_scope</code>	Valida alcance.
<code>_post_involves_main</code>	Comprueba si la cuenta participa.
<code>select_posts</code>	Filtra publicaciones.

Función o clase	Responsabilidad
determine_main_account	Elige cuenta principal.
_full_name_for_main	Nombre principal.
analyze_payloads	Pipeline principal.
_post_reference	Resumen de post extremo.
_post_table_row	Fila de tabla.
_follower_rows	Resumen de seguidores.
_recurrence_summary	Resumen de recurrencia.
analyze_posts	API compatible.

Backend app.py

Función o método	Responsabilidad
DashboardState.__init_ _	Inicializa estado.
replace_dataset	Reemplaza posts.
replace_payload	Reemplaza fuente validada.
replace_files	Carga lista de archivos.
clear	Limpia memoria.
current_bundle	Obtiene/recupera bundle.
analysis	Genera informe.
status	Estado público.
check_auth	Verifica credenciales.
authenticate	Respuesta 401.

Función o método	Responsabilidad
<code>require_login</code>	Middleware.
<code>_query_value</code>	Query string.
<code>_decode_payload</code>	JSON de bytes.
<code>_load_server_files</code>	Lee JSON del servidor.
<code>health</code>	Health check.
<code>status</code>	Ruta de estado.
<code>analysis</code>	Ruta principal.
<code>load_server_files</code>	Carga todos los archivos.
<code>sources</code>	Lista fuentes.
<code>analysis_guide</code>	Entrega guía.
<code>report_index</code>	Entrega la primera página.
<code>report_page</code>	Entrega la página HTML del nivel.
<code>plotly_asset</code>	Sirve Plotly local.
<code>serve_static</code>	Assets y portada.
<code>find_default_dataset</code>	Compatibilidad.
<code>make_handler</code>	Handler de pruebas.
<code>_parse_args</code>	CLI.

Frontend actual `web/dashboard.js`

Función	Responsabilidad
<code>apiJson</code>	Consulta la API y muestra errores.

Función	Responsabilidad
loadData	Carga el informe de la página actual.
renderShell	Activa la navegación y la posición del nivel.
renderHeader	Carga cuenta, alcance y cobertura.
barPlot / linePlot	Barras y líneas Plotly.
boxPlot / boxGroupsPlot	Distribuciones individual y comparada.
heatmapPlot	Matriz Plotly.
networkPlot	Red Plotly con nodos y aristas.
renderNarrative	Presenta el retrato de la cuenta.
renderResumen a renderFindings	Renderizan una página por nivel.
downloadPosts	Exporta publicaciones a CSV.

Frontend heredado web/app.js

Función	Responsabilidad
formatNumber	Miles.
formatCompact	Compacto.
formatPercent	Porcentaje.
escapeHtml	Escape HTML.
safeUrl	Seguridad de URLs.
normalizeText	Búsqueda.
formatDate	Fechas.
formatMonth	Meses.

Función	Responsabilidad
showToast	Notificaciones.
setLoading	Loading.
parseResponse	Errores API.
setUploadStatus	Estado de carga.
showUpload	Pantalla inicial.
showReport	Dashboard.
renderAll	Render general.
renderHeader	Encabezado.
renderKpis	Tarjetas.
renderDataInventory	Inventario.
renderPosts	Tabla y subgráficas.
chartSvg	Wrapper SVG.
gridLines	Ejes.
verticalBars	Barras.
lineChart	Líneas.
scatterPerformance	Dispersión.
renderPerformance	Render dispersión.
renderLikesByPost	Likes/post.
renderLikesTimeline	Timeline likes.
renderPostFrequency	Frecuencia.

Función	Responsabilidad
<code>renderLikesHistogram</code>	Histograma.
<code>renderBoxplot</code>	Boxplot.
<code>renderExceptions</code>	Excepciones.
<code>renderContentTypes</code>	Tipos.
<code>renderAudience</code>	Audiencia.
<code>renderConcentration</code>	Concentración.
<code>renderLorenz</code>	Lorenz.
<code>renderRecurrence</code>	Recurrencia.
<code>renderAudienceHeatmap</code>	Heatmap.
<code>networkColor</code>	Color de nodos.
<code>renderNetworkSvg</code>	SVG de red.
<code>renderAudienceNetwork</code>	Mapa audiencia.
<code>renderCollaborations</code>	Colaboraciones.
<code>renderNetworks</code>	Selector de red.
<code>renderEvolution</code>	Evolución.
<code>renderFindings</code>	Hallazgos.
<code>setExplanation</code>	Texto explicativo.
<code>renderExplanations</code>	Todas las explicaciones.
<code>loadAnalysis</code>	Recalcular.

Función	Responsabilidad
uploadFiles	Upload.
loadServerFiles	Carga del servidor.
downloadCsv	CSV de posts.
downloadNetworkCsv	CSV de red.
bindEvents	Eventos.
initialize	Arranque.

13. Ejemplo mental de una publicación

Supongamos este registro:

[illegible]

La aplicación interpreta:

- una publicación de `cuenta_principal`;
- 100 likes agregados;
- 20 comentarios reportados;
- dos comentarios capturados de Ana;
- Ana aparece con dos interacciones identificadas;
- Beto aparece como colaborador;

- Ana y Beto están en redes diferentes;
- una colaboración y dos comentarios pueden coincidir, pero no se afirma causalidad.

Este ejemplo muestra por qué una misma persona puede aparecer en varias filas y por qué es importante no llamar “likes de Ana” a los 100 likes de la publicación.

14. Mantenimiento futuro

Para agregar un nuevo tipo de evento, por ejemplo respuestas a comentarios:

1. Detectar el campo en `DatasetBundle`.
2. Crear un walker o una rama de normalización.
3. Añadirlo a la matriz de eventos.
4. Decidir si cuenta como interacción.
5. Actualizar cobertura.
6. Actualizar redes y audience.
7. Añadir su leyenda y explicación.
8. Documentarlo en esta guía.
9. Crear un fixture y una prueba.

Para agregar un nuevo gráfico:

1. Crear un contenedor con `id` en la página HTML correspondiente.
2. Añadir la explicación de qué muestra y qué no permite concluir.
3. Implementar una función que entregue trazas a `Plotly.newPlot`.
4. Llamarla desde la función `render...` del nivel.
5. Añadir leyenda si hay colores.
6. Comprobar que funciona sin datos.

Para agregar una página:

1. Crear el HTML en `web/pages/`.
2. Añadir su clave a `PAGE_FILES` y su entrada en `PAGE_ORDER/PAGE_META` del frontend.
3. Implementar su función de render.
4. Añadir el enlace anterior/siguiente y la navegación superior.
5. Probar la ruta `/informe/<page>` y el estado sin fuente.

15. Conclusión

La arquitectura fue diseñada para ser pequeña, transparente y explicable. Flask mantiene el estado, cada nivel tiene una página HTML propia y Plotly hace interactivas las gráficas. El sistema no intenta fabricar una verdad completa a partir de datos parciales. Separa:

- lo que Instagram o el scraper entregó;
- lo que la aplicación pudo identificar;
- lo que se calculó;
- lo que se observó;
- lo que no se puede concluir.

Esa separación es la base para que el informe sea útil para una persona no programadora y, al mismo tiempo, verificable por alguien que quiera revisar el código.